

**VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



Specialized Project (CO4029)

REPORT

**Designing a 2.5D Puzzle Game in Unity
Featuring a Time-loop Mechanic**

Lecturer:

Student: Nguyễn Trung Tín 2353187

HO CHI MINH CITY, SEPTEMBER 2026

Contents

1	Introduction	2
1.1	Project Background	2
1.2	Motivation	2
1.3	Project Challenges	2
1.4	Project Objectives	2
1.5	Development Approach	3
2	Background	3
2.1	Time-Loop and Time-Manipulation Games	3
2.2	Puzzle Game Design	4
2.3	Action Recording and Replay	4
2.4	State Management	4
2.5	Level Representation and Creation	4
2.6	Comparison with the Proposed Game	4
3	Implementation	5
3.1	Overall Game Architecture	5
3.2	Gameplay System	5
3.3	Action Recording System	6
3.4	Ghost Replay System	6
3.5	Room Reset and State Management	6
3.6	Puzzle Interaction System	6
3.7	Level Data and Level Loading	7
3.8	Level Creation Tool	7
3.9	Puzzle and Level Design	7

1 Introduction

1.1 Project Background

This project develops a 2.5D puzzle game in Unity. Each level is a room in which the player must complete an objective using a limited number of turns per attempt. The player may move, wait, interact with an object, or end the attempt. When the attempt ends – because the turn allowance is used, the player is defeated, or the player chooses to stop – the room returns to its initial state. The completed sequence of actions is retained and replayed by a ghost during the next attempt.

The player and ghosts occupy the same room and can affect its mechanisms. A ghost can, for example, repeat the actions that activate a switch while the current player uses the route that the switch opens. Further attempts can add more action sequences to the puzzle. The player is therefore solving the room across several attempts, rather than relying on a single uninterrupted run.

The project is organized into two phases. Phase 1 covers the core gameplay loop, a playable set of rooms, action recording and replay, reset behaviour, basic objectives and interactions, and a data-driven level workflow. Phase 2 is planned to add further mechanisms and hazards, progression, scoring, presentation improvements, and broader gameplay evaluation. This report describes the Phase 1 design and prototype.

1.2 Motivation

In a conventional restart, the player begins again with the same world state and only their own understanding has changed. Here, a restart also turns the previous attempt into an actor. An attempt that did not solve the room can still contribute: its ghost may hold a switch, move an object, or reach a useful position while the player follows a different plan.

This makes the action sequence itself part of the solution. The player must consider what to do now and what they want a future ghost to do. A short turn allowance gives each attempt a clear planning limit, while the room reset makes it possible to revise a plan and try again. The design aims to make experimentation useful by showing the consequences of an attempt in the next one.

1.3 Project Challenges

The main challenge is reliable and understandable replay. The game must record player commands and reproduce them consistently, including when an action has a different result in a later attempt. For example, a move that was possible before may be blocked by an object moved by another ghost. The rules for such cases, as well as the order in which players, ghosts, and room mechanisms act, must be defined clearly enough for players to plan around them.

Resetting the room presents a related state-management challenge. Room objects and mechanisms must return to their starting states at the end of an attempt, while the action histories needed by ghosts persist. A full level restart must also be distinguished from ending one attempt, since restarting a level should clear the histories built within it.

The project also needs a practical way to create and revise puzzle rooms. Each level must describe its layout, starting conditions, objectives, turn allowance, and interactive objects in data that the game can load. A basic level-creation workflow is included in the Phase 1 scope so that rooms can be iterated without implementing each one as a separate Unity scene.

1.4 Project Objectives

Phase 1 has the following objectives:

- review relevant time-loop, puzzle, and action-replay designs;
- define the gameplay loop, turn phases, reset rules, and system architecture;
- implement player movement, waiting, and interaction, with a turn limit for each attempt;
- record actions and replay completed attempts through ghosts;
- reset room state while preserving the histories required for replay;

- implement basic environmental interactions, objectives, and completion conditions;
- save and load level data in JSON and provide a basic level-creation workflow;
- integrate the systems into a playable prototype and produce a Game Design Document.

The number of attempts is not intended to be capped in the current design; the constraint is the turn allowance within each attempt. More advanced ghost coordination, hazards, hostile creatures, level progression, efficiency scoring, and expanded evaluation are planned for Phase 2.

1.5 Development Approach

Development begins by defining the action and reset rules, then implementing movement and interaction. Action recording and ghost replay are built on those rules, followed by room reset and basic mechanisms. Once the loop is in place, a set of rooms can introduce the mechanics gradually and combine them into puzzles that require cooperation across attempts. The JSON level format and creation workflow support iteration on this content.

The remainder of the report reviews related games and design concepts, describes the prototype's architecture and implementation, and presents deployment, evaluation, limitations, and conclusions.

2 Background

This section considers three uses of time in puzzle games: changing temporal rules, learning a recurring schedule, and carrying knowledge from one attempt to the next. It then reviews action replay, state management, and level authoring as design and implementation concerns related to this project.

2.1 Time-Loop and Time-Manipulation Games

Braid makes rewind part of its puzzle rules. Players use it to alter the relationship between movement and events in the level; some worlds also introduce objects or situations that do not follow the ordinary rewind rule. Solving a puzzle therefore requires reasoning about both position and which events can be reversed. Stamenković and Jačević analyze the game through the relations among time, space, motion, and causality.¹ Jonathan Blow's GDC presentation on *Braid*'s rewind system also describes the engineering requirement to reproduce past game states accurately while keeping a long rewind history within the memory limits of consoles.² These works make *Braid* relevant both as a puzzle-design example and as a technical case in temporal state restoration.

The Sexy Brutale uses a different structure. The events in its mansion recur on a shared daily schedule, and the player observes characters' routines before intervening to prevent their deaths. The developers describe the characters as following synchronized schedules across the mansion, with events in one location affecting what others hear or do elsewhere.³ Its central puzzle activity is to learn the timeline and identify where an intervention can change its outcome. The repeated setting makes observations useful: the player can compare what happens at the same point in successive cycles.

Twelve Minutes places the emphasis on knowledge retained by the player. Director Luis Antonio described accumulated knowledge as the concept behind the game and explained that short loops and a small environment help players understand the consequences of their actions and plan another attempt.⁴ The character and situation repeat, but the player uses what they have learned to choose different actions. This is relevant to the project's compact-room format and its aim of making attempts quick to inspect and revise.

¹Dušan Stamenković and Milan Jačević, *Time, Space, and Motion in Braid: A Cognitive-Semantic Approach to a Video Game*, *Games and Culture* 10, no. 2 (2015): 178–203, <https://doi.org/10.1177/1555412014557640>.

²Jonathan Blow, *The Implementation of Rewind in Braid*, Game Developers Conference, 2010, <https://www.gdcvault.com/play/1012850/The-Implementation-of-Rewind-in>.

³Cavalier Game Studios, *Murder Mystery Game The Sexy Brutale Available Now on Xbox One*, Xbox Wire, April 12, 2017, <https://news.xbox.com/en-us/2017/04/12/the-sexy-brutale-available-now-xbox-one/>.

⁴Bryant Francis, *Using Accumulated Player Knowledge to Drive Design Decisions in Twelve Minutes*, interview with Luis Antonio, *Game Developer*, September 9, 2021, <https://www.gamedeveloper.com/design/q-a-i-twelve-minutes-i-director-luis-antonio-on-making-good-time-loop-games>.

2.2 Puzzle Game Design

Puzzle design depends on players being able to understand the goal, the available actions, and the consequences of those actions. Schell and Adams discuss goals, rules, challenge, and feedback as core parts of game design.⁵ Applied to this project, the relevant principle is to introduce mechanics in a form players can observe, then combine familiar rules to create more demanding puzzles. A room should provide enough feedback for the player to understand what a ghost did and why an interaction succeeded or failed. Without that feedback, replay is difficult to reason about.

2.3 Action Recording and Replay

The Misadventures of P.B. Winterbottom provides a close example of recorded behaviour becoming part of a puzzle solution: the player records actions and cooperates with earlier versions of the character.⁶ This differs from a passive replay or visual trail because the recorded character can interact with the level alongside the current one.

The proposed game uses a related mechanic in turn-based rooms. Its design records player commands so a ghost can repeat the decisions that produced a useful position or interaction. The room rules then determine the outcome of those commands during replay. This means that replay must specify what happens when the room has changed since the original attempt—for example, when a move is blocked or an interaction target is no longer available. Defining those cases is part of making the ghost's behaviour predictable.

2.4 State Management

A loop reset has to restore the room without erasing the history needed by its ghosts. The level's initial object positions and mechanism states are resettable; completed action sequences persist across attempts. A full level restart has a wider scope and clears those histories as well. Keeping these categories distinct gives the game a clear basis for resetting the room and replaying earlier attempts.

Statecharts provide a formal way to describe system states and transitions, while Nystrom's programming-pattern examples discuss ways of organizing object behaviour and state in software.⁷ For this project, the practical implication is to define the order of a turn explicitly: collect an action, resolve its effects and environmental responses, then advance the turn. This helps prevent the result from depending on incidental update order.

2.5 Level Representation and Creation

A data-driven level separates puzzle content from the code that runs it. In this project, level data is intended to describe the room layout, starting conditions, objectives, turn allowance, and interactive objects; JSON provides the save and load format for the level workflow. Although procedural-content research also studies representations for game content, the project uses designed rooms rather than automatic level generation. The related concern is the need for a representation that the authoring workflow and game runtime can both interpret consistently.⁸

2.6 Comparison with the Proposed Game

These games provide distinct precedents rather than a single equivalent system. *Braid* makes temporal manipulation part of the puzzle rules; *The Sexy Brutale* asks players to learn and alter a recurring event schedule; and *Twelve Minutes* foregrounds knowledge accumulated across repeated attempts. *The Misadventures of P.B. Winterbottom* is a closer precedent for replayed actions cooperating with the current character.

The proposed game brings these ideas into a compact, turn-based room: each attempt has a limited action allowance, and past commands are replayed by ghosts that share the room with the player. The key distinction is that earlier attempts are not only remembered or used to alter time; they remain as actors whose actions can affect the current puzzle. This combination motivates the project's focus on consistent replay, clearly visible interactions, and a reliable separation between resettable room state and persistent action histories.

⁵Jesse Schell, *The Art of Game Design: A Book of Lenses*, 3rd ed. (CRC Press, 2019); Ernest Adams, *Fundamentals of Game Design* (New Riders, 2014).

⁶*The Misadventures of P.B. Winterbottom*, Steam store description, https://store.steampowered.com/app/40930/The_Misadventures_of_PB_Winterbottom/.

⁷David Harel, *Statecharts: A Visual Formalism for Complex Systems*, *Science* 231, no. 4737 (1987): 671–676; Robert Nystrom, *Game Programming Patterns* (Genever Benning, 2014).

⁸Noor Shaker, Julian Togelius, and Mark J. Nelson, *Procedural Content Generation in Games* (Springer, 2016).

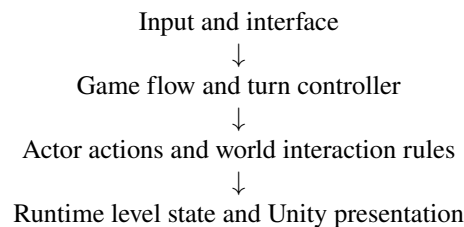
3 Implementation

This chapter describes the proposed architecture and implementation plan for the Phase 1 prototype. The design targets Unity and a turn-based, grid-based 2.5D puzzle game. The game world is presented in three dimensions, while movement and puzzle rules use a two-dimensional grid. This separation keeps the rules discrete and predictable while allowing the scene to use depth, lighting, and animation for presentation. Unless otherwise stated, the systems below describe the intended prototype design rather than completed implementation.

3.1 Overall Game Architecture

The prototype is divided into systems with separate responsibilities. The game-flow system manages level entry, attempts, completion, and restart. The turn controller coordinates the player, ghosts, and environment for each turn. Actor controllers produce actions, while world objects implement their own interaction rules. The level system loads and validates level data and constructs the room. The interface presents the current objective, remaining turns, and loop controls.

In Unity, scene objects can provide rendering, animation, and input, while the gameplay state is held in data-oriented runtime objects. This prevents visual effects from determining puzzle outcomes. For example, a player may animate between two grid cells, but the movement system changes the logical grid position according to the turn rules. A simplified dependency flow is:



The turn controller is the authority for advancing gameplay. Other systems may request an action or report a result, but they do not independently advance the turn counter. This provides one place to enforce turn limits and the order of events. The level data, current room state, and ghost histories are treated as separate data: level data defines the initial room, room state changes during an attempt, and ghost histories persist between attempts within the same level.

3.2 Gameplay System

The main game states are level loading, active attempt, attempt reset, level completion, and level restart. After a level loads, the game starts an attempt with the player at the defined starting cell and the room in its initial state. On each turn, the player chooses one action: move, wait, or interact. Existing ghosts replay the actions recorded during their respective attempts. The environment then resolves mechanisms and hazards before the game advances to the next turn.

Each turn is divided into four phases:

1. **Turn start:** the controller checks the attempt state and prepares one action for each ghost that still has a recorded action, along with the player's next action.
2. **Action:** the player supplies a move, wait, or interaction command. Ghost commands are read from their action histories for the same turn index.
3. **Results:** actions are resolved, then mechanisms and hazards respond to the updated room. The game checks for player defeat and level completion.
4. **Turn end:** the player's command is added to the current history, the turn counter advances, and the controller checks the turn limit.

The turn limit applies to normal player actions. Ending an attempt early is a loop-control command and does not need to consume an additional turn. An attempt ends when the player chooses to end it, reaches the turn limit, or is defeated. The action history up to that point is kept as a ghost history, after which the room resets for the next attempt. Completing the level instead enters the completion state. Restarting the level clears all ghost histories created in that level and reloads its initial state.

3.3 Action Recording System

The action recorder stores one command per player turn. A command contains an action type and only the data needed to describe the player's intent. For example, a move stores a direction, while an interaction stores the stable identifier of its target. The recorder does not need to store rendered positions or animation frames: these are presentation details and can be reproduced from the logical action results.

An action history is an ordered sequence indexed from the first turn of an attempt. At the end of an attempt, the sequence is copied into the level's ghost-history collection. Histories are retained across ordinary attempt resets and removed when the player deletes the newest ghost or restarts the level. The current attempt has its own temporary history, which is committed as a ghost when that attempt ends. This distinction avoids modifying an existing ghost while the current player is acting.

Recording commands rather than outcomes allows a ghost to respond to the current room according to the normal rules. A command that succeeded during the original attempt may fail during replay if an object now blocks the way. The game should show this result clearly, while keeping the command sequence unchanged. This makes the room state and the action history sufficient to explain replay behaviour.

3.4 Ghost Replay System

At the start of each turn, every ghost reads the command at the current turn index from its action history. When a history has no command for that index, the ghost waits for the rest of the attempt. Ghosts therefore replay in parallel with the current player, and a history shorter than the current attempt does not stop the ghost from remaining in the room.

To make the outcome deterministic, actors are processed in a fixed order. The prototype can resolve existing ghosts from oldest to newest and then resolve the current player. Movement and object pushes are resolved first, followed by interactions and environmental effects. A blocked move becomes a no-op; an invalid interaction also has no effect. Ghosts can occupy the same cell as one another and the player, so they do not block actor movement. They can still activate mechanisms. Solid world objects, walls, and closed doors continue to block movement according to their rules.

This resolution policy is intentionally simple for Phase 1. If two actors attempt to push the same object, the earlier actor in the fixed order is resolved first; later actions are checked against the resulting state and may fail. The rule can be revised if playtesting shows that this ordering is difficult to understand, but it must remain explicit and visible in the game feedback.

3.5 Room Reset and State Management

The level definition is immutable during play and serves as the source for the initial room state. Runtime state contains changing values such as entity positions, door states, lever states, and hazard activation. At the beginning of a level, the level system instantiates the runtime entities from the definition. At the end of each attempt, it discards or reinitializes that runtime state and creates the room again from the same definition.

Ghost histories are maintained outside the resettable room state. An ordinary attempt reset preserves completed histories, clears the current attempt history and turn counter, and restores all level objects. Deleting the most recent ghost removes its history and then performs the same room reset. A full level restart also clears all histories. Separating these operations gives each reset a clear scope and avoids trying to reverse every object change individually.

3.6 Puzzle Interaction System

Each interactive object responds to a small set of gameplay events through defined rules. A box may be pushed into an unoccupied cell, a door may block movement while closed, and a lever may change the state of one or more connected objects. The level data defines object properties and links, while shared interaction logic resolves whether an action is valid. Stable object identifiers are used for references such as a lever controlling a door.

The initial implementation should support movement, waiting, direct interaction, boxes, switches or levers, and doors. Additional planned objects include lasers, turrets, crevices, traps, mirrors, and conveyors. These can be added as separate object behaviours while using the same turn phases. For example, a laser checks its line after movement and mechanism updates; a mirror changes the beam direction; and a conveyor requests movement for entities on its cells. Hazards should be evaluated in a consistent order so that the same room state and commands always give the same outcome.

The game should provide feedback when an action changes the room or fails. Examples include a door opening, a box being blocked, or a ghost attempting to interact with a missing or inactive target. This feedback is essential to the puzzle design because the player needs to understand which part of a replay affected the current attempt.

3.7 Level Data and Level Loading

Levels are stored as JSON files so that puzzle content can be changed without changing gameplay code. A level definition contains a format version, identifier, display name, grid dimensions, turn allowance, player starting position, exit or completion rule, tile layout, and interactive entities. The coordinate system uses integer grid positions. The Unity scene maps those positions to world coordinates for rendering.

The following fragment illustrates the intended form of a level file:

```
{
  "version": 1,
  "id": "room_01",
  "size": { "width": 8, "height": 6 },
  "turnLimit": 12,
  "playerStart": { "x": 1, "y": 4 },
  "completion": { "type": "reachExit", "x": 7, "y": 1 },
  "objects": [
    { "id": "box_a", "type": "box", "x": 2, "y": 4 },
    { "id": "lever_a", "type": "lever", "x": 4, "y": 2,
      "controls": ["door_a"] },
    { "id": "door_a", "type": "door", "x": 6, "y": 2,
      "initialState": "closed" }
  ]
}
```

The loader reads the file, checks that its version is supported, and validates required fields before constructing the room. Validation should reject invalid dimensions, coordinates outside the grid, duplicate identifiers, missing references, and missing required elements such as a player start or completion condition. If validation fails, the game should report the level identifier and the specific data error to help the designer correct the file. A successful load creates the board, places tiles and objects, saves the initial-state definition for resets, and starts the first attempt.

3.8 Level Creation Tool

The Phase 1 level-creation workflow is intended primarily for development and testing. A basic grid editor can be implemented as a Unity editor tool. It should allow a designer to set the grid dimensions and turn limit, place tiles and objects, set the player start and exit, edit object properties, and save or load the level as JSON. A validation command should identify missing or conflicting data before the level is played.

The first version does not require a polished public-facing editor. Its main purpose is to shorten the iteration cycle between changing a room and testing its puzzle. The editor and runtime loader should use the same level format and validation rules so that a level accepted by the editor can be loaded by the game. In the absence of an editor interface, the JSON format also permits levels to be created or revised directly as text files.

3.9 Puzzle and Level Design

Levels should introduce one rule at a time before combining rules. Early rooms can teach movement, the turn allowance, and the effect of an attempt becoming a ghost. Subsequent rooms can require a ghost to activate a switch while the player moves through a door, then introduce movable boxes and environmental hazards. Later rooms can combine several histories and mechanisms, requiring the player to plan both the current attempt and the actions that future ghosts will repeat.

Each room should make its objective and relevant mechanisms readable. The player needs to see how many turns remain, which ghosts are active, and which mechanisms changed during replay. Optional performance measures, such as the number of attempts or total actions, can be added after the core puzzle rules are stable. The current design does not cap the number of attempts; the per-attempt turn limit supplies the primary constraint.



The level set should be evaluated for both puzzle solvability and replay clarity. A room may be technically solvable but frustrating if the player cannot infer why a ghost's action failed or how a mechanism changed. Iteration should therefore check that mechanics are introduced gradually, failed actions are communicated, and each additional ghost contributes a discernible effect to the solution.